DRISHTI AI TIMELINE

Day	You (Perception)	Siya (Behaviour + Objects)	Kavya (Integration + Dashboard)
Day 1	Setup env + multi-person detection + tracking + basic pose estimation	Define suspicious behaviours with thresholds + start phone/paper object detection	Project structure + config + basic video pipeline + log format + privacy plan
Day 2	Improve tracking + extract temporal features (head/body/hand) + real-time optimisation	Implement behaviour rules + link objects to students + add confidence scores	Event logging system + explainable alert format + dashboard skeleton
Day 3	Full perception pipeline (detect→track→pose→features) in real-time	Connect behaviour engine + object detection to live pipeline + generate alerts	Connect everything to live dashboard (video + alerts + logs)
Day 4	Robustness testing (lighting, angles, occlusion) + demo clips	Reduce false positives + final behaviour tuning + example clips	Analytics module + downloadable logs + UI polish + privacy enforcement
Day 5	Final FPS optimisation + demo videos + help in recording	Behaviour explanation examples + documentation section	Final dashboard + README + demo script + dry-run support

Day 1 – Foundation & Setup

You (Perception Core)

- Set up development environment (Python, OpenCV, Ultralytics YOLO, MediaPipe or RTMPose, ByteTrack/DeepSORT)
- Collect / prepare 3–4 sample multi-person seated examination videos (different angles if possible)
- Implement multi-person detection using YOLOv8
- Add multi-object tracker (ByteTrack recommended) so each student gets a stable ID
- Integrate pose estimation (YOLOv8-Pose or MediaPipe Pose) on every tracked person
- Output for each frame: Student ID + Bounding box + Keypoints (at least 17)
- Test on one video and save a short output video with boxes + skeletons

Siya (Behaviour + Objects)

- Write a clear list of suspicious behaviours with measurable definitions:
    - Repeated sideward glances
    - Excessive head turning
    - Body rotation toward neighbouring students
    - Unusual / repetitive hand movements
    - Possible non-verbal communication attempts
- Decide thresholds (e.g., head yaw > 40° for > 2 seconds, repeated 3+ times in 10 seconds, etc.)
- Start object detection baseline (YOLOv8 pretrained or custom) for Mobile Phone and Paper/Chit
- Create a simple function that takes keypoints of one person and returns current head orientation & hand positions

Kavya (Integration + Dashboard)

- Create clean project folder structure and GitHub repo
- Write config.yaml (camera source, model paths, thresholds, etc.)
- Build basic video reading + display pipeline (OpenCV or Streamlit)
- Design the event log format (timestamp, student_id, behaviour_type, confidence, explanation, keypoints_snapshot)
- Decide privacy approach: store only anonymised Student IDs + keypoints (no face images)
- Create empty modules: detector.py, tracker.py, pose.py, behaviour.py, object_detector.py, alert.py, logger.py, dashboard.py

End of Day 1 Goal: Working detection + tracking + pose on sample video + clear behaviour definitions + project skeleton ready.

Day 2 – Core Modules Development

You

- Improve tracker robustness (handle short occlusions, reduce ID switches)
- Extract temporal features for each student:
    - Head yaw/pitch history (last 30–60 frames)
    - Shoulder / hip rotation
    - Hand keypoint trajectories
- Calculate movement speed and frequency of head turns
- Make the whole perception pipeline run in real-time (target ≥ 15–20 FPS on your machine)
- Add simple visualisation: Student ID + skeleton + basic head direction arrow

Siya

- Implement rule-based behaviour engine using the temporal features from You
- Code each suspicious behaviour as a separate detector function

- Link detected objects (phone / paper) to the nearest student ID
- Add confidence score for every alert
- Create a simple test script that runs behaviour rules on saved pose data
- Start collecting examples of false positives (normal thinking, stretching, looking at paper)

Kavya

- Implement the event logging system (save to JSON + CSV)
- Design the exact format of an explainable alert Example: "Student 4 – Repeated sideward glance (left) 4 times in 9 seconds"
- Build Streamlit (or Gradio) dashboard skeleton with:
    - Live video placeholder
    - Active alerts panel
    - Event log table
- Create a mock alert generator so the dashboard can be tested independently

End of Day 2 Goal: Behaviour rules working offline + object detection running + logging + dashboard skeleton ready.

Day 3 – Full Integration (Most Important Day)

All three sit together for at least 3–4 hours

You

- Connect detection → tracking → pose → feature extraction into one clean real-time pipeline
- Expose a simple API/function that returns current state of all students (ID, keypoints, features)
- Fix any real-time bottlenecks
- Handle multiple students cleanly (10+ if possible)

Siya

- Plug the behaviour engine and object detector into the live pipeline
- Generate real-time alerts with explanations
- Tune thresholds on live/sample videos to reduce obvious false positives
- Make sure every alert clearly states the observed behaviour (as required in the PPT)

Kavya

- Connect the live pipeline to the dashboard
- Show live video with overlays (boxes, optional skeletons, student IDs)
- Display real-time alerts with explanations
- Show scrolling event log
- Add basic controls (start/stop, select video source)

End of Day 3 Goal: Complete end-to-end real-time system running with live alerts and logs.

Day 4 – Robustness, Analytics & Polish

You

- Test and improve performance under different conditions (lighting changes, different camera angles, denser seating)
- Add basic occlusion handling notes / improvements
- Optimise FPS further if needed
- Prepare 2–3 good demo video clips (normal behaviour + suspicious behaviour)

Siya

- Major focus on reducing false positives (add "normal behaviour" filters)
- Improve temporal logic (e.g., require sustained or repeated action)
- Finalise object detection association with students
- Prepare clear examples of each suspicious behaviour for the demo/presentation

Kavya

- Build behavioural analytics module:
    - Number of alerts per student
    - Timeline of events
    - Simple activity summary
- Make event logs downloadable
- Strengthen privacy (ensure no face data is saved)
- Polish the dashboard UI so it looks professional and invigilator-friendly
- Add a small "System Status" section (FPS, number of students tracked, etc.)

End of Day 4 Goal: System is robust enough, analytics ready, UI polished, all PPT requirements covered.

Day 5 – Testing, Demo Preparation & Documentation

Morning – Joint Testing (All three)

- Run the system on multiple videos / live camera
- Test edge cases: occlusion, people standing up, normal movements, actual suspicious actions
- Note down remaining limitations honestly
- Fix only critical bugs

Afternoon – Individual Final Tasks

You

- Final performance check and FPS optimisation
- Prepare clean demo videos with overlays
- Help record the final demo

Siya

- Prepare a short list of detected behaviours with explanations (for judges)
- Create 4–5 clear before/after or example clips showing the system catching suspicious actions
- Write the "How behaviour detection works" section for documentation

Kavya

- Final dashboard polish + smooth demo flow
- Write README + short architecture diagram
- Prepare the demo script (who speaks what)
- Export sample event logs and analytics screenshots

Evening

- Full dry-run of the demo as a team (at least twice)
- Decide speaking order and backup plans

Final Coverage Check (All PPT points assigned)

- Detect & track multiple seated students → **You**
- Extract pose landmarks & movement patterns → **You**
- Identify suspicious head/body/hand movements → **Siya**
- Identify objects (Mobile Phone, Paper/Chit…) → **Siya**
- Real-time alerts with interpretable explanations → **Siya + Kavya**
- Behavioural analytics & event logs → **Kavya**
- Privacy-conscious monitoring → **Kavya** (enforced by all)
- Robustness + minimising false alarms → **All (focus Day 4)**